C10 DevOps and cost

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track C: Microsoft Fabric 2
C10 DevOps and cost 3
C10-L1 Git integration for Fabric workspaces 3
C10-L2 Deployment pipelines, dev to test to production 7
C10-L3 Capacity, smoothing and throttling 11
Track C best-practice checklist 16
Appendix: solutions and answer keys 17

Track C: Microsoft Fabric

Unify data, analytics and Copilot in Fabric, and verify what Copilot produces.

C10 DevOps and cost

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Git integration, deployment pipelines and capacity monitoring.

C10-L1 Git integration for Fabric workspaces

Concept

Git integration connects a Fabric workspace to a branch and folder in a Git repository. Supported providers are Azure DevOps, GitHub and GitHub Enterprise, all cloud-based only. Some item types are still in preview, so check the supported-items list for each one you use (Microsoft Learn, checked September 2026).

What Git holds. Item definitions, not data. Each item gets its own folder of definition files. For a lakehouse, Git tracks metadata only: its name, shortcuts and OneLake security roles. Tables and files are not tracked, and Git operations never overwrite them. Rebuilding a workspace from Git gives you empty lakehouses. Your pipelines must reload the data.

Two directions:

- **Commit** exports workspace changes to the branch.
- **Update** imports branch changes into the workspace and overwrites the workspace version.

The source control panel shows each item's state. If an item changed in both places, you have a **conflict** to resolve before you continue.

Who does what. Only a workspace **Admin** connects or disconnects the workspace. Members and Contributors commit and update, limited by their permissions in the Git repository. Viewers see no Git information.

One workspace, one branch. A workspace is a shared runtime. Anything one person changes there affects everyone using it. Microsoft's recommended pattern is a **separate workspace per developer**, each connected to its own feature branch. You develop in your workspace, commit to your branch, and open a pull request into the main branch. The shared workspace connected to main then updates from Git.

Alternatively, develop in client tools such as VS Code for notebooks or Power BI Desktop for reports, and push to your branch.

Keep notebooks small and parameterised, and keep configuration in files your pipeline reads. Reviewers read diffs. A 900-line notebook with hard-coded paths produces diffs nobody can review.

Why it matters

Three engineers shared one development workspace connected to main. One of them updated from Git while another had an hour of uncommitted notebook edits. The update overwrote the workspace version, and the edits were lost. With a workspace and branch per developer, each person's work stays isolated until it merges.

Worked example: weak code

```
1 Workspace: sales_dev, connected to main, used by 3 engineers
2 Process: edit in sales_dev, commit to main when it works
3 Recovery plan: restore the lakehouse from Git
```

1. **Line 1.** A shared runtime: one person's update overwrites another's work.
2. **Line 2.** No review before main.
3. **Line 3.** Git holds lakehouse metadata only. Tables and files are not in Git.

Worked example: strong code

```
1 Workspaces: sales_dev_ana (branch feature/ana-dedupe), sales_dev_raj
    (branch feature/raj-gold)
2 Process: commit to your feature branch, pull request into main, review,
    merge
3 Shared: sales_integration connected to main, updated from Git after each
    merge
3 Recovery plan: redeploy items from Git, then rerun ingestion pipelines to
    reload data
```

1. **Line 1.** One workspace per developer, one branch per workspace.
2. **Line 2.** Changes reach main through review.
3. **Line 4.** Items come from Git. Data comes from pipelines.

One workspace per developer on its own branch, pull requests into main, and a data reload plan, because Git holds definitions, not data.

Recap card

- **What Git integration stores for a lakehouse** Metadata only: name, shortcuts, OneLake security roles. Not tables or files.
- **Commit vs Update** Commit exports workspace changes to Git. Update imports Git changes and overwrites the workspace version.

- **Who connects a workspace to Git?** Admin only. Members and Contributors commit and update. Viewers see no Git information.
- **Recommended development process** One workspace per developer on its own branch. Pull requests into main.

Practice

Exercise 1. Multiple choice (level 1)

A lakehouse is connected to Git. What does the repository hold for it?

- a) The Delta tables and files
- b) Metadata only, such as its name, shortcuts and OneLake security roles
- c) Nothing: lakehouses are not supported
- d) A full backup, restorable to any commit

Why?

- r1) Microsoft Learn: only lakehouse metadata is tracked. Git operations never overwrite table or file data.
- r2) Git has a size limit that excludes tables.

Exercise 2. Multiple choice (level 2)

You click **Update** in a workspace. A notebook has an uncommitted edit in the workspace and a different change in Git. What does Fabric report?

- a) A conflict you must resolve
- b) Nothing: Git wins silently
- c) Nothing: the workspace wins silently
- d) The two versions merge automatically

Why?

- r1) An item changed in both the workspace and the branch is a conflict, shown in the source control panel.
- r2) Fabric merges notebooks cell by cell.

Exercise 3. Multiple choice (level 2)

Which workspace role connects a workspace to a Git repository?

- a) Admin
- b) Member
- c) Contributor
- d) Viewer

Why?

- r1) Connect and disconnect are Admin operations. Members and Contributors commit and update.
- r2) Only Admins have Git accounts.

Exercise 4. Write the function (level 3)

Your team's rule for the source control panel: resolve conflicts first, then commit your changes, then update from Git. Write `next_step(statuses: dict[str, str]) -> str`. Values are `"synced"`, `"uncommitted"`, `"update_required"` or `"conflict"`. Return `"resolve"` if any item is in conflict, else `"commit"` if any is uncommitted, else `"update"` if any needs an update, else `"none"`. Raise `ValueError` for an unknown status. Use a constant for the allowed statuses.

Your code must pass these checks:

- Conflicts come before everything else.
- Commit your changes before you update, or the update overwrites them.
- Update when only Git changed. Nothing to do when everything is synced.
- Unknown statuses raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
ALLOWED_STATUSES = ...

# Next source control action for a workspace.
def next_step(statuses: dict[str, str]) -> str: ...
```

Starter code:

```
def next_step(statuses):
    if "uncommitted" in statuses.values():
        return "commit"
    return "update"
```

Exercise 5. Architecture design (level 4)

Set up source control for a team of four engineers building notebooks and pipelines in Fabric.

Options: One workspace per engineer, each on its own feature branch, One shared dev workspace connected to main.

1. Development workspaces
2. Path to main

3. After a workspace is rebuilt from Git

Sources

- Providers Azure DevOps, GitHub, GitHub Enterprise (cloud only); some item types preview; workspace structure and subfolders preserved. (verified: https://learn.microsoft.com/en-us/fabric/cicd/git-integration/intro-to-git-integration (updated 2026-07-21))
- Commit exports and overwrites Git content; update overwrites workspace content; Admin connects; Member/Contributor commit and update per Git role; Viewer sees no Git info. (verified: https://learn.microsoft.com/en-us/fabric/cicd/git-integration/git-integration-process (updated 2026-06-15))
- A workspace connects to a single branch; recommended separate workspace per developer (branched workspace) or client tools. (verified: https://learn.microsoft.com/en-us/fabric/cicd/git-integration/manage-branches (updated 2026-06-15))
- Lakehouse Git tracks metadata only (name, shortcuts, data access roles); tables and files not tracked and never overwritten. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/lakehouse-git-deployment-pipelines (updated 2026-05-08))
- A conflict occurs when the same item changes in both the workspace and the repository; Commit is disabled until it is resolved. (verified: https://learn.microsoft.com/en-us/fabric/cicd/git-integration/conflict-resolution (updated 2026-06-15))

C10-L2 Deployment pipelines, dev to test to production

Concept

Git integration (lesson C10-L1) versions your items. **Deployment pipelines** move them between environments. A pipeline has **2 to 10 stages**, with Development, Test and Production as the default three. Each stage is a workspace (Microsoft Learn, checked September 2026).

What a deployment copies. When you deploy from one stage to the next, Fabric copies item content and metadata to the target stage. It does not copy **data**. After deploying a semantic model, you refresh it in the target stage. After deploying a lakehouse, you load its tables there.

Overwrite, not replace. If the item already exists in the target stage, Fabric overwrites its content. The item's **ID, URL and permissions stay the same**, so reports and links in production keep working.

Deployment rules give each stage its own settings:

- **Data source rules**: point a semantic model at the production database, not the test one.
- **Parameter rules**: set a parameter's value per stage.
- **Default lakehouse rules**: attach a notebook to the stage's own lakehouse.

You define a rule in the **target** stage, on the item. It applies from the next deployment into that stage.

Autobinding. If a deployed item depends on another item, such as a report on a semantic model, and the linked item exists in the target stage, Fabric binds to the target copy automatically.

Compare shows which items differ between two stages before you deploy. Use it every time. The deployment pipelines REST APIs automate the same steps from a build system.

Many teams combine the tools. Developers work in Git-connected workspaces and merge through pull requests. The merged branch updates the Development stage, and deployment pipelines promote tested content to Test and Production. Microsoft's CI/CD guidance also describes Git-only release options, so pick one and document it.

Why it matters

A team deployed a notebook from Test to Production. It still pointed at the test lakehouse, so for a week the production job wrote its gold tables into Test. The dashboard in Production read stale tables and nobody noticed. A default lakehouse rule on the notebook in the Production stage would have attached it to the production lakehouse on every deployment.

Worked example: weak code

```
1 Stages: Development, Production
1 Notebook nb_gold: default lakehouse lh_test (hard-coded)
3 Semantic model sales_model: data source test SQL database
2 Before deploying: nothing
```

1. **Line 2.** Production runs against test data.
2. **Line 4.** No compare step: you learn what changed after it breaks.

Worked example: strong code

```
1 Stages: Development, Test, Production
1 Rule in Production on nb_gold: default lakehouse = lh_prod
3 Rule in Production on sales_model: data source = production SQL database
```

```
❷ Before deploying: compare Test with Production, review the differing items
❸ After deploying: refresh sales_model, run the load pipeline
```

1. **Line 2.** Rules live in the target stage and apply on every deployment.
2. **Line 4.** Know what changes before it changes.
3. **Line 5.** Deployment copies items, not data.

Stage-specific settings belong in deployment rules. Compare before you deploy. Reload data after, because deployments copy items, not data.

Recap card

- **Deployment pipeline stages** 2 to 10 stages, default Development, Test, Production. One workspace per stage.
- **What a deployment copies** Item content and metadata. Not data: refresh models and reload tables after.
- **Deployment rule types** Data source, parameter, default lakehouse. Defined in the target stage on the item.
- **Deploying over an existing item** Content is overwritten. ID, URL and permissions stay the same.

Practice

Exercise 1. Multiple choice (level 1)

You deploy a semantic model from Test to Production for the first time. What must you do next?

- a) Nothing: the data is copied with it
- b) Refresh the model in Production, because data is not copied between stages
- c) Recreate the report permissions
- d) Reconnect the workspace to Git

Why?

- r1) Microsoft Learn: a semantic model refresh is required because data isn't copied from one stage to another.
- r2) Deployment pipelines always clear data in Production.

Exercise 2. Multiple choice (level 2)

Where do you define a rule that points a notebook at `lh_prod` in the Production stage?

- a) In the Production stage, on the notebook, as a default lakehouse rule

- b) In the Development stage, on the notebook
- c) In the notebook's code, with an if statement on the workspace name
- d) In the Git repository

Why?

- r1) Rules are defined in the target stage on the item, and apply from the next deployment into that stage.
- r2) Rules are global to the pipeline.

Exercise 3. Put in order (level 2)

Order a release from Test to Production.

- Check row counts and a known report figure in Production
- Check the deployment rules in the Production stage
- Compare Test with Production and review differences
- Deploy Test to Production
- Refresh semantic models and run load pipelines in Production

Exercise 4. Write the function (level 3)

Model deployment rules. Write `deployed_settings(source: dict[str, str], rules: dict[str, dict[str, str]], stage: str) -> dict[str, str]`. `source` holds an item's settings in the source stage. `rules` maps a stage name to the settings its rules override. Return the settings in `stage`: the source values, with that stage's rules applied. Do not modify `source`. Raise `ValueError` if a rule names a setting the item does not have.

Your code must pass these checks:

- Apply the target stage's rules over the source values.
- A stage without rules keeps the source values.
- Do not modify the source settings: return a new dict.
- A rule for a setting the item lacks is a configuration error.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Item settings in a target stage after its deployment rules apply.
def deployed_settings(
    source: dict[str, str],
    rules: dict[str, dict[str, str]],
    stage: str,
) -> dict[str, str]: ...
```

Starter code:

```python
def deployed_settings(source, rules, stage):
    source.update(rules[stage])
    return source
```

Exercise 5. Architecture design (level 4)

Design the release process for a team that uses Git integration and wants a Test stage signed off before Production.

Options: Workspace connected to main, updated from Git after each merged pull request, Engineers edit directly in the Development stage workspace.

1. Development stage
2. Promotion to Test and Production
3. After each Production deployment

Sources

- Deployment pipelines have 2 to 10 stages, default three. (verified: https://learn.microsoft.com/en-us/fabric/cicd/deployment-pipelines/intro-to-deployment-pipelines (updated 2026-07-17))
- Data isn't copied between stages, semantic model refresh required; existing items overwritten with ID, URL and permissions unchanged; autobinding to linked items in the target stage; REST APIs available. (verified: https://learn.microsoft.com/en-us/fabric/cicd/deployment-pipelines/understand-the-deployment-process (updated 2025-12-15))
- Rule types: data source, parameter, default lakehouse; defined in the target stage; apply from the next deployment. (verified: https://learn.microsoft.com/en-us/fabric/cicd/deployment-pipelines/create-rules (updated 2025-12-15))
- CI/CD options include Git-based release options and deployment pipelines; many organisations use a hybrid. (verified: https://learn.microsoft.com/en-us/fabric/cicd/manage-deployment (updated 2026-03-18))

C10-L3 Capacity, smoothing and throttling

Concept

Every Fabric operation draws capacity units (CUs) from its workspace's capacity (lesson C1-L1). Fabric lets operations **burst** above the capacity's size, then **smooths** the cost over future time (Microsoft Learn, checked September 2026).

Smoothing. Fabric measures usage in 30-second **timepoints**, 2,880 per day.

- **Interactive** operations, such as report queries, are smoothed over at least 5 minutes and up to 64 minutes.
- **Background** operations, such as scheduled Spark jobs, pipeline runs and model refreshes, are smoothed over **24 hours**.

Example: an F2 has 2 CUs, or 48 CU hours per day. A background job that uses 1 CU hour adds 1 / 48, about 2.1%, to every timepoint for the next 24 hours.

Throttling starts when smoothed usage borrows too much future capacity:

Future capacity already used	What happens
Up to 10 minutes	Overage protection: no throttling
Over 10, up to 60 minutes	Interactive jobs are delayed by 20 seconds
Over 60 minutes, up to 24 hours	Interactive jobs are rejected
Over 24 hours	All requests are rejected, background too

Background jobs keep running well after users feel the delay. A capacity deep in debt takes hours to recover, because only idle capacity pays it back.

Surge protection lets capacity admins reject background jobs earlier, before the capacity reaches deep throttling. At workspace level, it caps a workspace's share of CU use over a rolling 24 hours, unless you tag the workspace **Mission Critical**.

The Capacity Metrics app shows what happened. A capacity admin installs it and shares it. The **Health** page flags throttling and rejections across your capacities. The **Compute** page gives 14 days of utilisation by item and operation.

Practical rules: schedule heavy background jobs away from business hours, keep production on its own capacity, and check the Compute page before you buy a larger SKU.

Why it matters

A team scheduled a full rebuild of 40 gold tables at 08:00 on an F16. By 09:30 the capacity had borrowed more than an hour of future use, and Power BI reports started failing for every user. The rebuild itself kept running. Moving it to 02:00, and making it incremental, kept the morning reports responsive without a larger SKU.

Worked example: weak code

```
❶ Capacity: F16, shared by production reports and data science experiments
❷ Schedule: full rebuild of gold tables at 08:00 on weekdays
❸ Monitoring: users report slow dashboards
```

1. **Line 1.** Experiments draw from the same CUs as production.
2. **Line 2.** Heavy background work just before peak interactive use.
3. **Line 3.** You find out from users, after throttling has started.

Worked example: strong code

```
  1 Capacity: F16 for production only. Experiments on a separate F4
❶ Schedule: incremental gold build at 02:00
❷ Surge protection: background rejection threshold set on the F16
❸ Monitoring: Capacity Metrics app Health page checked daily, Compute page
     reviewed weekly
```

1. **Line 2.** Less work, smoothed over hours when few users are active.
2. **Line 3.** Background jobs are stopped before users are rejected.
3. **Line 4.** You see throttling in the data before users feel it.

Know which of your jobs are background, run them off-peak, protect production with surge protection, and read the Metrics app before you resize.

Recap card

- **Smoothing windows** Interactive: 5 to 64 minutes. Background: 24 hours. Timepoints are 30 seconds, 2,880 a day.
- **Throttling stages** Up to 10 min: none. 10 to 60: 20 s interactive delay. 60 min to 24 h: interactive rejection. Over 24 h: all rejected.
- **Surge protection** Capacity level: earlier background rejection. Workspace level: cap a workspace's CU share over 24 hours, except Mission Critical.
- **Capacity Metrics app** Installed by a capacity admin. Health page for throttling. Compute page for 14 days of usage.

Practice

Exercise 1. Multiple choice (level 1)

Over what period does Fabric smooth the CU cost of a background Spark job?

- a) 24 hours
- b) 5 minutes
- c) The job's own duration
- d) It is not smoothed

Why?

- r1) Microsoft Learn: background operations are smoothed over a 24-hour period.
- r2) Spark jobs run for exactly 24 hours.

Exercise 2. Multiple choice (level 2)

A capacity has used 90 minutes of future capacity. What happens to a user opening a Power BI report, and to a scheduled pipeline?

- a) The report is rejected. The pipeline still runs
- b) Both run with a 20-second delay
- c) Both are rejected
- d) Both run normally

Why?

- r1) Between 60 minutes and 24 hours, interactive jobs are rejected. Background rejection starts only above 24 hours.
- r2) Pipelines always have priority.

Exercise 3. Fill the gap (level 2)

A background job uses 2 CU hours on an F4. Complete the smoothing calculation.

```
F4 capacity per day: 4 CUs x 24 hours = _____(1) CU hours
Share of every timepoint for 24 hours: 2 / _____(1) = about _____(2)%
```

Exercise 4. Write the function (level 3)

Write `throttle_stage(future_minutes: float) -> str` returning the throttling stage for minutes of future capacity already used: `"none"` up to 10, `"interactive_delay"` above 10 up to 60, `"interactive_rejection"` above 60 up to 1,440 (24 hours), and `"background_rejection"` above that. Raise `ValueError` for negative input. Keep the limits in constants.

Your code must pass these checks:

- Four bands: 10 minutes, 60 minutes, 24 hours.
- Each limit is inclusive: exactly 10 minutes is still overage protection.
- Negative usage is invalid.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
OVERAGE_PROTECTION_MINUTES = ...
```

```
3
4  # Module constant.
5  INTERACTIVE_DELAY_MINUTES = ...
6
7  # Module constant.
8  INTERACTIVE_REJECTION_MINUTES = ...
9
10 # Fabric throttling stage for the future capacity already consumed.
11 def throttle_stage(future_minutes: float) -> str: ...
```

Starter code:

```
1  def throttle_stage(future_minutes):
2      if future_minutes < 10:
3          return "none"
4      if future_minutes < 60:
5          return "interactive_delay"
6      return "interactive_rejection"
```

Exercise 5. Architecture design (level 4)

Your F32 throttles every weekday between 08:30 and 10:00. The Compute page shows a 06:00 full refresh of a large semantic model and data science notebooks running all day on the same capacity.

Options: Make it incremental and move it to 01:00, Keep it: refreshes are background and smoothed.

1. The 06:00 full refresh
2. Data science notebooks
3. Protection for report users

Sources

- Timepoints 30 s, 2,880 per day; interactive smoothed 5 to 64 minutes; background 24 hours; F2 1 CU hour example ~2.1%; throttling stages at 10 min, 60 min, 24 h with 20 s interactive delay. (verified: https://learn.microsoft.com/en-us/fabric/enterprise/throttling (updated 2026-08-14))
- Surge protection: capacity-level background rejection threshold; workspace-level CU cap over rolling 24 h with Mission Critical exclusion. (verified: https://learn.microsoft.com/en-us/fabric/enterprise/surge-protection (updated 2026-05-27))
- Capacity Metrics app installed by capacity admin; Health page; Compute page 14-day view. (verified: https://learn.microsoft.com/en-us/fabric/enterprise/metrics-app (updated 2026-01-14))

Track C best-practice checklist

☐ Check the status (GA or preview) of every Fabric feature before you depend on it.

☐ One copy of each dataset. Use shortcuts or mirroring before you copy.

☐ Lakehouse for Spark and files. Warehouse for T-SQL writes and multi-table transactions.

☐ The SQL analytics endpoint is read-only for table data. Write through Spark or a Warehouse.

☐ Semantic models use star schemas. DAX measures state their filter context.

☐ Direct Lake models are checked for fallback to DirectQuery.

☐ Every Copilot output is read, run on known data and compared against a second method.

☐ Workspaces map to domains. Access uses roles and OneLake security, not item sharing.

☐ Items live in Git. Changes move through deployment pipelines.

☐ Capacity use is monitored, and heavy jobs are scheduled off peak.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

C10-L1 Git integration for Fabric workspaces

C10-L1 exercise 1

Hints. (1) Does Git store data or definitions? (2) Definitions. (3) Tables are not tracked.

Answer. b) Metadata only, such as its name, shortcuts and OneLake security roles Because: r1) Microsoft Learn: only lakehouse metadata is tracked. Git operations never overwrite table or file data.

Git versions your items. Pipelines rebuild your data.

Common wrong answers.

- `a`: Tables and files are not tracked in Git.
- `c`: Lakehouses are supported. Only their metadata is tracked.
- `d`: Git is not a data backup. Plan data recovery separately.
- `b|r2`: Right answer, wrong reason. Git integration tracks item definitions by design, not data.

C10-L1 exercise 2

Hints. (1) Both sides changed. (2) What does the source control panel show then? (3) A conflict.

Answer. a) A conflict you must resolve Because: r1) An item changed in both the workspace and the branch is a conflict, shown in the source control panel.

Commit often, and work in your own workspace, so conflicts stay rare.

Common wrong answers.

- `b`: Changes on both sides are flagged as a conflict first.
- `c`: The workspace does not win by default.
- `d`: There is no automatic merge. You resolve the conflict.
- `a|r2`: Right answer, wrong reason. Fabric does not merge notebooks. You choose how to resolve.

C10-L1 exercise 3

Hints. (1) Connecting changes the workspace itself. (2) Lesson C1-L3. (3) Admin.

Answer. a) Admin Because: r1) Connect and disconnect are Admin operations. Members and Contributors commit and update.

Admins connect. Members and Contributors commit and update, within their Git permissions.

Common wrong answers.

- `b`: Members commit and update. Connecting is for Admins (lesson C1-L3).
- `c`: Contributors commit and update. Connecting is for Admins.
- `d`: Viewers see no Git information.
- `a|r2`: Right answer, wrong reason. Git permissions matter for everyone. The workspace role decides who connects.

C10-L1 exercise 4

Hints. (1) Collect the set of statuses present. (2) Validate against the constant first. (3) Check in priority order: conflict, uncommitted, update_required.

Answer.

```python
ALLOWED_STATUSES = frozenset({"synced", "uncommitted", "update_required", "conflict"})


def next_step(statuses: dict[str, str]) -> str:
    """Next source control action for a workspace.

    Args:
        statuses: Item name to source control status.

    Returns:
        "resolve", "commit", "update" or "none".

    Raises:
        ValueError: If a status is not recognised.
    """
    present = set(statuses.values())
    unknown = present - ALLOWED_STATUSES
    if unknown:
        raise ValueError(f"unknown statuses: {sorted(unknown)}")
    if "conflict" in present:
        return "resolve"
    if "uncommitted" in present:
        return "commit"
```

```
    if "update_required" in present:
        return "update"
    return "none"
```

Priority order encodes the safe sequence: never let an update overwrite work you have not committed.

C10-L1 exercise 5

Hints. (1) Where does work in progress live? (2) How do changes reach main? (3) What does Git hold for a lakehouse?

Answer.

- One workspace and branch per engineer isolates work in progress.
- Pull requests put a review between each change and main.
- Git holds lakehouse metadata only, so data comes back through pipelines.

Isolated workspaces, reviewed merges, and pipelines that rebuild data.

Common wrong answers.

- workspaces = `shared`: A shared workspace is a shared runtime. One person's update overwrites another's uncommitted work.
- merge = `direct`: Direct commits skip review.
- data = `nothing`: Tables and files are not tracked in Git.

C10-L2 Deployment pipelines, dev to test to production

C10-L2 exercise 1

Hints. (1) What does a deployment copy? (2) Items and metadata. (3) Not data.

Answer. b) Refresh the model in Production, because data is not copied between stages Because: r1) Microsoft Learn: a semantic model refresh is required because data isn't copied from one stage to another.

Deploy, then refresh or reload in the target stage.

Common wrong answers.

- `a`: Data is not copied between stages.
- `c`: Deployment does not touch permissions. On later deployments, IDs, URLs and permissions stay the same.
- `d`: Deployment pipelines and Git connections are separate.
- `b|r2`: Right answer, wrong reason. Deployment does not clear data. It never copied any.

C10-L2 exercise 2

Hints. (1) Which stage needs the different value? (2) The target. (3) Default lakehouse rule.

Answer. a) In the Production stage, on the notebook, as a default lakehouse rule Because: r1) Rules are defined in the target stage on the item, and apply from the next deployment into that stage.

The target stage owns its settings.

Common wrong answers.

- `b`: Rules belong to the target stage.
- `c`: Branching on workspace names hard-codes environments into code. Use a deployment rule.
- `d`: Deployment rules are pipeline configuration, set per stage.
- `a|r2`: Right answer, wrong reason. Rules are per stage and per item, not global.

C10-L2 exercise 3

Hints. (1) Rules must exist before deployment. (2) Compare before deploy. (3) Data after deploy, checks last.

Answer.

1. Check the deployment rules in the Production stage
2. Compare Test with Production and review differences
3. Deploy Test to Production
4. Refresh semantic models and run load pipelines in Production
5. Check row counts and a known report figure in Production

Rules, compare, deploy, reload, verify.

Common wrong answers.

- Rules apply at deployment. Check them before you deploy.
- Verify after the data is loaded, or you check an empty model.

C10-L2 exercise 4

Hints. (1) `rules.get(stage, {})` handles stages without rules. (2) Compare rule keys with source keys. (3) `{**source, **stage_rules}` builds a new dict.

Answer.

```python
def deployed_settings(
    source: dict[str, str], rules: dict[str, dict[str, str]], stage: str
) -> dict[str, str]:
```

```
    """Item settings in a target stage after its deployment rules apply.

    Args:
        source: Settings in the source stage.
        rules: Stage name to the settings its rules override.
        stage: Target stage.

    Returns:
        A new dict of settings for the target stage.

    Raises:
        ValueError: If a rule names a setting the item does not have.
    """
    stage_rules = rules.get(stage, {})
    unknown = set(stage_rules) - set(source)
    if unknown:
        raise ValueError(f"rules for unknown settings: {sorted(unknown)}")
    return {**source, **stage_rules}
```

Settings per stage, validated, with the source left intact: the same contract deployment rules give you.

C10-L2 exercise 5

Hints. (1) How do changes reach Development? (2) Which tool keeps IDs and applies rules? (3) What does a deployment not copy?

Answer.

- Git and pull requests control what reaches Development.
- Deployment pipelines promote with per-stage rules and a compare step.
- Deployments copy items, not data, so reload and check.

Git for development, deployment pipelines for promotion, and a reload and check after each release.

Common wrong answers.

- dev = `edits`: Direct edits skip review and conflict with Git updates.
- promote = `copy`: Manual copies lose rules, compare and autobinding, and produce new IDs and URLs.
- post = `done`: Data is not copied between stages.

C10-L3 Capacity, smoothing and throttling

C10-L3 exercise 1

Hints. (1) Background or interactive? (2) Background jobs are long and heavy. (3) 24 hours.

Answer. a) 24 hours Because: r1) Microsoft Learn: background operations are smoothed over a 24-hour period.

Background work is spread over a day, so one heavy job raises every timepoint a little.

Common wrong answers.

- `b`: Five minutes is the minimum for interactive operations.
- `c`: Smoothing spreads cost beyond the job's run time.
- `d`: Background operations are smoothed, over 24 hours.
- `a|r2`: Right answer, wrong reason. The smoothing window is fixed. It does not depend on job length.

C10-L3 exercise 2

Hints. (1) Which band is 90 minutes in? (2) 60 minutes to 24 hours. (3) Interactive rejection.

Answer. a) The report is rejected. The pipeline still runs Because: r1) Between 60 minutes and 24 hours, interactive jobs are rejected. Background rejection starts only above 24 hours.

Users feel throttling first. Background jobs, often the cause, keep running.

Common wrong answers.

- `b`: The 20-second delay applies between 10 and 60 minutes.
- `c`: Background jobs are rejected only above 24 hours of future use.
- `d`: Above 60 minutes, interactive jobs are rejected.
- `a|r2`: Right answer, wrong reason. Background jobs have a later threshold, not priority.

C10-L3 exercise 3

Hints. (1) CUs times hours in a day. (2) Divide the job by the daily total. (3) 96 CU hours, about 2.1%.

Answer.

```
F4 capacity per day: 4 CUs x 24 hours = 96 CU hours
Share of every timepoint for 24 hours: 2 / 96 = about 2.1%
```

2 / 96 is about 2.1%, the same share as the F2 example with half the job.

Common wrong answers.

- g1 = `4`: Multiply CUs by 24 hours for the daily CU hours.

C10-L3 exercise 4

Hints. (1) Three constants: 10, 60 and 1,440 minutes. (2) Use `<=` for each limit. (3) Validate first.

Answer.

```python
OVERAGE_PROTECTION_MINUTES = 10
INTERACTIVE_DELAY_MINUTES = 60
INTERACTIVE_REJECTION_MINUTES = 24 * 60


def throttle_stage(future_minutes: float) -> str:
    """Fabric throttling stage for the future capacity already consumed.

    Args:
        future_minutes: Minutes of future capacity used by smoothed operations.

    Returns:
        "none", "interactive_delay", "interactive_rejection" or "background_rejection".

    Raises:
        ValueError: If future_minutes is negative.
    """
    if future_minutes < 0:
        raise ValueError("future_minutes must not be negative")
    if future_minutes <= OVERAGE_PROTECTION_MINUTES:
        return "none"
    if future_minutes <= INTERACTIVE_DELAY_MINUTES:
        return "interactive_delay"
    if future_minutes <= INTERACTIVE_REJECTION_MINUTES:
        return "interactive_rejection"
    return "background_rejection"
```

Boundaries are where bugs hide. Test each limit exactly.

C10-L3 exercise 5

Hints. (1) Where does a 06:00 job's cost land? (2) Who shares the capacity? (3) Which setting protects interactive users?

Answer.

- Smoothing spreads a 06:00 job over the next 24 hours, including the morning peak. Less work, earlier, helps.
- Separate capacities stop experiments from consuming production CUs.
- Surge protection rejects background work before users are rejected. Resize only if the Compute page still shows a shortfall.

Reduce and move background work, isolate experiments, then protect users with surge protection.

Common wrong answers.

- refresh = `keep`: Smoothing does not remove cost. It spreads it across the morning peak.
- ds = `same`: All-day notebooks add to every timepoint on the production capacity.
- protect = `resize`: Fix the workload first. A larger SKU doubles the bill and hides the cause.